

UNIVERSIDAD DEL VALLE DE GUATEMALA

Computación paralela y distribuida

Facultad de Ingeniería

Sección 21

Santiago Solorzano Padilla



Hoja de trabajo 3

https://github.com/jaq23369/Hoja_de_trabajo3_Paralela.git

Joel Jaquez - 23369

Kevin Villagran - 23584

GUATEMALA, 30 de septiembre 2026

Ejercicio 1: Utilizando diferentes 2, 4 y 6 procesos

Capturas:

- 2 procesos:

```
joel@MacBook-Pro-de-Joel-2 Incisos % mpicc -isysroot "$(xcrun --show-sdk-path)" HT3_inciso1.c -o HT3_inciso1
ld: warning: building for macOS-26.2, but linking with dylib '/opt/homebrew/opt/open-mpi/lib/libmpi.40.dylib' which was built for
newer version 27.0
joel@MacBook-Pro-de-Joel-2 Incisos % mpirun -np 2 ./HT3_inciso1
Proceso 0: Oficina Central
Proceso 1: Sucursal 1
Proceso 0: Existen 2 procesos en la simulacion.
Proceso 1: Existen 2 procesos en la simulacion.
joel@MacBook-Pro-de-Joel-2 Incisos %
```

- 4 procesos:

```
joel@MacBook-Pro-de-Joel-2 Incisos % mpirun -np 4 ./HT3_inciso1
Proceso 3: Sucursal 3
Proceso 3: Existen 4 procesos en la simulacion.
Proceso 1: Sucursal 1
Proceso 1: Existen 4 procesos en la simulacion.
Proceso 2: Sucursal 2
Proceso 2: Existen 4 procesos en la simulacion.
Proceso 0: Oficina Central
Proceso 0: Existen 4 procesos en la simulacion.
joel@MacBook-Pro-de-Joel-2 Incisos %
```

- 6 procesos:

```
joel@MacBook-Pro-de-Joel-2 Incisos % mpirun -np 6 ./HT3_inciso1
Proceso 4: Sucursal 4
Proceso 4: Existen 6 procesos en la simulacion.
Proceso 1: Sucursal 1
Proceso 1: Existen 6 procesos en la simulacion.
Proceso 0: Oficina Central
Proceso 0: Existen 6 procesos en la simulacion.
Proceso 5: Sucursal 5
Proceso 5: Existen 6 procesos en la simulacion.
Proceso 3: Sucursal 3
Proceso 3: Existen 6 procesos en la simulacion.
Proceso 2: Sucursal 2
Proceso 2: Existen 6 procesos en la simulacion.
joel@MacBook-Pro-de-Joel-2 Incisos %
```

Preguntas:

- **¿Qué representa MPI_COMM_WORLD dentro de un programa MPI?**
 - MPI_COMM_WORLD es el grupo que se crea automáticamente cuando el programa llama a MPI_Init(). En ese grupo están todos los procesos que se lanzan con mpirun, dentro del grupo cada proceso tiene su propio número que se llama rank y va desde 0 hasta la cantidad de procesos menos uno. Gracias a ese número los procesos se pueden reconocer entre ellos y saber a quién mandarle información.
- **¿Qué ventaja proporciona usar MPI_COMM_WORLD como comunicador inicial en lugar de manejar manualmente una lista de procesos?**
 - La ventaja principal es que MPI arma este grupo solo cuando inicia el programa así que no hay que hacer nada manual, es decir que no hace falta crear una lista de procesos, ni ponerle un número a cada uno, ni saber en qué parte de la computadora está corriendo cada proceso. Con MPI_rank() cada proceso sabe cuál es su número y con MPI_Comm_size() sabe cuántos procesos hay en total, por esa razón el programa del inciso 1 funcionó igual con 2, 4 y 6 procesos sin cambiar nada del código. Ahora si la lista se manejara manualmente habría que modificar cada vez que cambia la cantidad de procesos y sería mucho más fácil cometer errores.
- **En una ejecución local sobre una sola computadora, ¿por qué los procesos MPI siguen considerándose independientes aunque físicamente utilicen la misma máquina?**
 - Aunque todos los procesos corren en la misma computadora para el sistema operativo cada uno es un programa separado, es decir que cada proceso tiene su propia memoria y sus propias variables, entonces si un proceso cambia el valor de una variable los demás no se dan cuenta. La única forma de que se compartan información es mandándose mensajes con las funciones de MPI, además el sistema operativo reparte los procesos entre los núcleos del procesador y cada uno avanza a su propio ritmo. Esto se aprecia mejor en las capturas de arriba.

Ejercicio 2. Comunicación directa entre dos procesos:

- a. **Identifica dentro de las instrucciones MPI_Send() y MPI_Recv(), utilizadas en el programa HT3_inciso2.c, qué valor fue asignado a cada uno de sus parámetros**

MPI_Send manda un mensaje desde un proceso hacia otro, y MPI_Recv lo recibe. En la tabla se muestran los valores que usa el programa HT3_inciso2.c en la llamada original, la que envía las ventas.

Parámetro de MPI_Send	Valor utilizado	Parámetro de MPI_Recv	Valor utilizado
buffer	&ventas	buffer	&ventas
count	1	count	1
datatype	MPI_FLOAT	datatype	MPI_FLOAT
destination	0	source	1
tag	100	tag	100
communicator	MPI_COMM_WORLD	communicator	MPI_COMM_WORLD
		status	MPI_STATUS_IGNORE

- b. **Modifica el programa HT3_inciso2.c para que la Sucursal 1 envíe a la Oficina Central un segundo dato: la cantidad de pedidos procesados durante el día.**

Código modificado en /Incisos/HT3_inciso2.c

```
user@LAPTOP-BBJPVM9E:/mnt/c/Users/kevin/OneDrive/Escritorio/Archivos/TAREAS/PARALELA/HOJA 3/Repo/Incisos$ mpirun -np 2 ./HT3_inciso2
Sucursal 1: ventas del día = Q1250.75
Sucursal 1: pedidos procesados = 48
Sucursal 1: reporte enviado a Oficina Central.
Oficina Central: reporte recibido.
Ventas reportadas por Sucursal 1: Q1250.75
Pedidos procesados por Sucursal 1: 48
```

- c. **Preguntas de análisis**

- i. **¿Por qué se utilizaron tag diferentes para el envío de ventas y pedidos?**

Porque el tag sirve para identificar de qué trata cada mensaje. Con el tag 100 la Oficina Central sabe que el mensaje es de ventas y con el tag 200 sabe que es de pedidos. Así cada MPI_Recv toma justo el mensaje que le corresponde.

- ii. **¿Qué ocurriría si ambos mensajes fueran enviados con el mismo tag y la Oficina Central realizara dos recepciones consecutivas?**

Los dos mensajes saldrían del mismo proceso, hacia el mismo destino y con el mismo tag. MPI entrega esos mensajes en el orden en que se enviaron, entonces el primer MPI_Recv recibiría las ventas y el segundo los pedidos. Funcionaría solo mientras el orden de los envíos y de las recepciones sea exactamente el mismo. Si el orden cambiara, un float se leería como int, o al revés, y los valores quedarían mal. Con tags distintos el programa no depende de ese orden.

- iii. **¿Qué relación debe existir entre los parámetros utilizados en MPI_Send() y los parámetros correspondientes de MPI_Recv() para que la comunicación sea correcta?**

El destination del envío debe ser el rank del proceso que recibe, y el source de la recepción debe ser el rank del proceso que envía. El tag y el communicator deben ser

iguales en los dos. El datatype debe ser el mismo, y el count del MPI_Recv debe ser igual o mayor que el del MPI_Send para que el mensaje quepa en el buffer.

iv. Si la Oficina Central cambiara el source del segundo MPI_Recv() a un proceso distinto de rank = 1, ¿qué efecto tendría sobre la ejecución?

La Oficina Central esperaría un mensaje de un proceso que nunca lo envía. El mensaje de pedidos viene del rank 1, entonces nunca coincidiría con la recepción. El rank 0 se quedaría bloqueado esperando y el programa no terminaría. Si además se ejecuta con 2 procesos y el source es 2 o más, ese rank ni siquiera existe y MPI marcaría un error.

v. En este ejercicio se utilizan dos llamadas a MPI_Send() y dos a MPI_Recv(). ¿Qué ventaja tiene mantener ambos datos como mensajes separados en lugar de enviarlos como un único dato?

Cada dato conserva su propio tipo, las ventas son float y los pedidos son int, y en un solo mensaje habría que empaquetarlos o convertirlos a un mismo tipo. Además la Oficina Central puede recibir cada dato por separado y en el orden que quiera, y el código queda más fácil de leer porque cada mensaje tiene un solo significado.

Ejercicio 3: Difusión de información a todos los procesos

- a. Identifica dentro de la instrucción `MPI_Bcast()` utilizada en el programa `HT3_inciso3.c` qué valor fue asignado a cada uno de sus parámetros.

i.

Parámetro	Valor utilizado en el programa	¿Qué representa para la empresa?
buffer	<code>&precio</code>	La dirección de la variable precio. En la Oficina Central guarda el precio que se va a mandar y en las sucursales es donde se guarda el precio que les llega.
count	1	Se manda un solo dato, el nuevo precio.
datatype	<code>MPI_FLOAT</code>	El precio es un número con decimales (Q25.50).
root	0	La Oficina Central es la que tiene el precio y la que lo manda a todos.
communicator	<code>MPI_COMM_WORLD</code>	El precio se manda a toda la empresa, o sea a la oficina central y a todas las sucursales.

- b. Modifica el programa `HT3_inciso3.c` para que la Oficina Central comunique a todas las sucursales, además del nuevo precio, un porcentaje de descuento que deberá aplicarse durante el día.

i.

```
joel@MacBook-Pro-de-Joel-2 Incisos % mpicc -isysroot "$(xcrun --show-sdk-path)" HT3_inciso3.c -o HT3_inciso3
ld: warning: building for macOS-26.2, but linking with dylib '/opt/homebrew/opt/open-mpi/lib/libmpi.40.dylib' which was built for newer version 2
7.0
joel@MacBook-Pro-de-Joel-2 Incisos % mpirun -np 6 ./HT3_inciso3
Oficina Central: nuevo precio = Q25.50
Oficina Central: descuento = 10.00 %
Sucursal 2: precio recibido = Q25.50
Sucursal 2: descuento recibido = 10.00 %
Sucursal 4: precio recibido = Q25.50
Sucursal 1: precio recibido = Q25.50
Sucursal 1: descuento recibido = 10.00 %
Sucursal 4: descuento recibido = 10.00 %
Sucursal 5: precio recibido = Q25.50
Sucursal 5: descuento recibido = 10.00 %
Sucursal 3: precio recibido = Q25.50
Sucursal 3: descuento recibido = 10.00 %
joel@MacBook-Pro-de-Joel-2 Incisos %
```

c. Preguntas

i. ¿Qué función cumple el parámetro root dentro de MPI_Bcast()?

- El parámetro root indica cuál proceso tiene el dato original y es el encargado de mandarlo a los demás. En este programa root vale 0 entonces la Oficina Central es la que reparte el precio. Para el proceso root la variable precio es lo que se manda y para los demás procesos esa misma variable es el lugar donde se guarda el valor que les llega.

ii. ¿Por qué solamente el proceso rank 0 necesita asignar inicialmente un valor a la variable precio?

- Porque rank 0 es el root, entonces es el único que manda el precio. Cuando las sucursales ejecutan MPI_Bcast(), el valor que tenían en su variable precio se reemplaza por el valor que mandó la Oficina Central. Así que no importa qué valor tenían antes al final todas quedan con el mismo precio, esto también tiene sentido en la empresa porque solo la oficina central decide los precios y las sucursales solo los reciben.

iii. ¿Qué diferencia existe entre utilizar MPI_Bcast() para distribuir el precio y realizar múltiples llamadas a MPI_Send() desde la Oficina Central?

- Con MPI_Bcast() basta una sola instrucción que ejecutan todos los procesos. Con MPI_Send() la Oficina Central tendría que hacer un ciclo y mandarle el precio a cada sucursal una por una, y cada sucursal tendría que hacer su propio MPI_Recv(). Eso es más código y es más fácil equivocarse con el source, el destination o el tag. Además, con MPI_Send() todo el trabajo de mandar los mensajes lo hace la Oficina Central sola. MPI_Bcast() puede repartir mejor ese trabajo, porque las sucursales que ya recibieron el precio pueden ayudar a pasarlo a otras. Cuando hay muchas sucursales eso hace que el precio llegue más rápido a todas.

iv. ¿Qué ocurriría si en una de las sucursales se especificara un valor de root diferente al utilizado por los demás procesos en la misma llamada a MPI_Bcast()?

- Las reglas de MPI piden que todos los procesos usen el mismo root en la misma llamada a MPI_Bcast(). Si una sucursal pone otro root, el programa queda en un error y el resultado no está definido. Por ejemplo si la Sucursal 2 pone root igual a 2 cree que ella es la que manda el precio entonces no recibe el valor de la Oficina Central y en su lugar intenta mandar su propia variable que ni siquiera tiene un valor asignado. Lo más probable es que el programa se quede trabado esperando un mensaje que nunca llega, o que algunas sucursales terminen con un precio equivocado.

v. ¿Por qué todos los procesos que pertenecen a MPI_COMM_WORLD deben participar en la ejecución de MPI_Bcast()?

- Porque MPI_Bcast() es una operación colectiva, o sea que funciona solo si todos los procesos del comunicador la ejecutan. La función no termina hasta que cada proceso hace su parte, ya sea mandar el dato o recibirlo. Si una sucursal no llama a MPI_Bcast() esa sucursal nunca recibe el precio y los demás procesos se pueden quedar esperando haciendo que el programa se trabe. En la empresa sería como mandar un aviso a todas las sucursales y que una nunca lo lea así que esa sucursal seguiría trabajando con un precio viejo.

vi. Después de ejecutar MPI_Bcast(), ¿qué valor contiene la variable precio en cada uno de los procesos?

- Después de MPI_Bcast() la variable precio vale 25.50 en todos los procesos, incluyendo la Oficina Central que conserva el valor que ella misma definió. Cada proceso tiene su propia copia de la variable en su propia memoria así que todos tienen el mismo valor pero guardado por separado. Esto se puede ver en la ejecución, donde cada sucursal muestra el precio recibido como Q25.50.

Ejercicio 4. Distribución del trabajo:

- a. **Identifica dentro de la instrucción `MPI_Scatter()` utilizada en el programa `HT3_inciso4.c` qué valor fue asignado a cada uno de sus parámetros.**

`MPI_Scatter` toma un arreglo del proceso root, lo parte en bloques del mismo tamaño y entrega un bloque a cada proceso, en orden de rank. En la tabla se muestran los valores del programa original de `HT3_inciso4.c` y su significado en la simulación.

Parámetro	Valor utilizado	Qué representa en la simulación
sendbuf	pedidos	El arreglo de 4 enteros que tiene la Oficina Central con los pedidos de cada ubicación. Solo importa en el rank 0.
sendcount	1	La cantidad de elementos que se manda a cada proceso. Cada ubicación recibe un pedido.
sendtype	<code>MPI_INT</code>	Los pedidos son números enteros.
recvbuf	<code>&pedido_recibido</code>	La variable donde cada ubicación guarda el dato que le tocó.
recvcount	1	La cantidad de elementos que cada proceso espera recibir.
recvtype	<code>MPI_INT</code>	El dato recibido también es un entero.
root	0	El proceso que reparte los datos, la Oficina Central.
communicator	<code>MPI_COMM_WORLD</code>	El grupo con la Oficina Central y las tres sucursales.

- b. **Modifica el programa `HT3_inciso4.c` para que la Oficina Central distribuya dos datos a cada ubicación en lugar de uno.**

```
user@LAPTOP-BBJPVM9E:/mnt/c/Users/kevin/OneDrive/Escritorio/Archivos/TAREAS/PARALELA/HOJA 3/Repo/Incisos$ mpicc HT3_inciso4.c -o HT3_inciso4
user@LAPTOP-BBJPVM9E:/mnt/c/Users/kevin/OneDrive/Escritorio/Archivos/TAREAS/PARALELA/HOJA 3/Repo/Incisos$ mpirun -np 4 ./HT3_inciso4
Oficina Central: distribuyendo pedidos...
Oficina Central: 120 pedidos asignados, 6 empleados disponibles.
Sucursal 1: 95 pedidos asignados, 5 empleados disponibles.
Sucursal 2: 140 pedidos asignados, 8 empleados disponibles.
Sucursal 3: 110 pedidos asignados, 7 empleados disponibles.
user@LAPTOP-BBJPVM9E:/mnt/c/Users/kevin/OneDrive/Escritorio/Archivos/TAREAS/PARALELA/HOJA 3/Repo/Incisos$ |
```

- c. **Preguntas de análisis**

- i. **¿Qué diferencia existe entre `sendbuf` y `recvbuf` dentro de `MPI_Scatter()`?**

El `sendbuf` es el arreglo de origen, el que tiene todos los datos a repartir, y solo se usa en el proceso root. El `recvbuf` es el lugar donde cada proceso guarda su parte, y todos los procesos lo usan, incluido el root.

- ii. **¿Qué determina el parámetro `sendcount` y cómo afecta la cantidad de datos que recibe cada proceso?**

El `sendcount` es la cantidad de elementos que se manda a cada proceso. Si vale 1, cada proceso recibe un elemento. Si vale 2, recibe dos. El arreglo del root debe tener

sendcount por la cantidad de procesos elementos, en este caso 2 por 4, o sea 8.

iii. En el programa base, ¿qué parte del arreglo pedidos recibe cada proceso y qué relación tiene esto con su rank?

Cada proceso recibe la posición del arreglo igual a su rank. El rank 0 recibe pedidos[0] (120), el rank 1 recibe pedidos[1] (95), el rank 2 recibe pedidos[2] (140) y el rank 3 recibe pedidos[3] (110). MPI reparte los bloques en orden, empezando por el rank 0.

iv. ¿Por qué solamente el proceso definido como root necesita contener inicialmente todos los datos que serán distribuidos?

Porque el root es el único que reparte. Los demás procesos solo reciben su parte en el recvbuf y MPI ignora su sendbuf. Por eso no tienen que guardar ni llenar el arreglo completo, y se ahorra memoria.

v. Si el arreglo del proceso root contiene menos datos de los necesarios para la cantidad de procesos y el valor de sendcount, ¿qué problema podría producirse?

MPI leería memoria que está fuera del arreglo. Por ejemplo, con 4 procesos y sendcount 2 se necesitan 8 elementos, y si el arreglo tiene 4, los últimos procesos recibirían valores basura. En el peor caso el programa se cae por acceso inválido a memoria. El resultado no se puede predecir.